

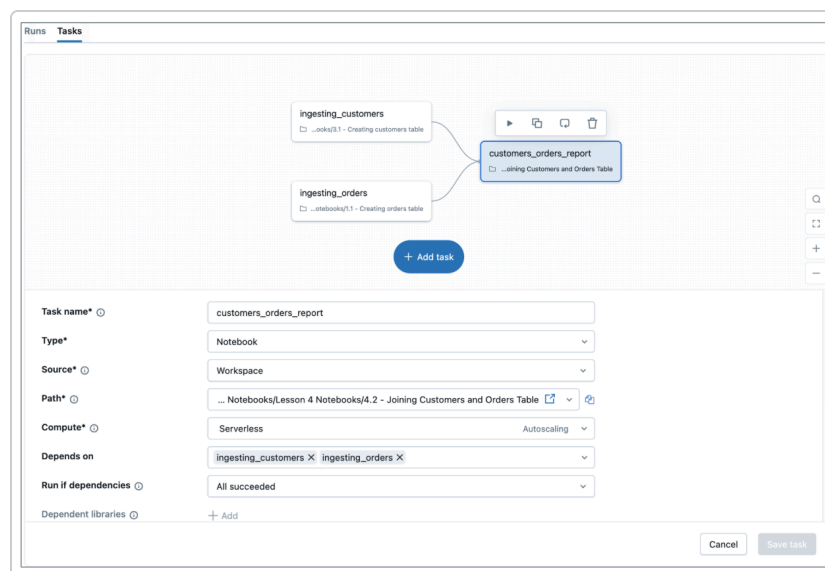
# Grundlagen: Jobs, Tasks und Komponenten

Section 4 · Working with Lakeflow Jobs · Quelle: Kurs 2, Kapitel 2–4

Sobald Daten in Databricks aufgenommen und transformiert werden sollen, braucht es eine Instanz, die diese Arbeitsschritte zuverlässig plant, ausführt und überwacht. Genau das leistet **Lakeflow Jobs** – die Orchestrierungskomponente von Lakeflow. Sie ist das Bindeglied zwischen einzelnen Notebooks, SQL-Abfragen oder Pipelines und einem verlässlichen, automatisierten Produktivbetrieb. Dieses Dokument erklärt die Grundbausteine: **Jobs**, **Tasks** und deren Anordnung als **DAG** (Directed Acyclic Graph).

## 1. Jobs und Tasks: eine hierarchische Beziehung

Ein **Job** ist die übergeordnete Ressource in Databricks, mit der Datenverarbeitung, ETL-Läufe, Analysen oder Machine-Learning-Workloads geplant, koordiniert und ausgeführt werden. Ein Job besteht aus einem oder mehreren **Tasks** – das sind die eigentlichen Arbeitseinheiten, die jeweils eine konkrete Aufgabe übernehmen, zum Beispiel das Ausführen eines Notebooks, eines Skripts oder einer SQL-Abfrage. Diese Beziehung ist hierarchisch: Ein Job ist der Container, die Tasks darin sind die Bausteine, die die eigentliche Arbeit verrichten.



*Ein Job kann aus einem oder mehreren Tasks bestehen, die gemeinsam einen Workflow bilden.*

## 2. Task-Typen und unterstützte Sprachen

Für jeden Task lässt sich der passende Task-Typ wählen. Databricks unterstützt unter anderem folgende Typen:

- **Notebook** – führt ein Databricks-Notebook in Python, SQL, Scala oder R aus.
- **SQL** – führt eine SQL-Datei oder eine gespeicherte Abfrage auf einem SQL-Warehouse aus.
- **Pipeline** – startet eine deklarative Lakeflow-Pipeline (Streaming Tables/Materialized Views).
- **Python-Skript / Python-Wheel** – führt eigenständigen bzw. paketierte Python-Code aus.
- **dbt, JAR (Java), Spark Submit, AI/BI-Dashboard** und weitere Typen für Legacy- oder BI-Anwendungsfälle.

Diese Vielfalt erlaubt es, praktisch jede Art von Workload innerhalb eines einzigen Jobs zu orchestrieren – unabhängig davon, in welcher Sprache oder mit welchem Werkzeug ein Team arbeitet. Jeder Task-Typ bringt eigene Konfigurationsoptionen mit: Pfad zum Code, benötigte Bibliotheken, Parameter, Benachrichtigungen und Retry-Verhalten lassen sich pro Task individuell einstellen.

### 3. Rechenressourcen (Compute) pro Task

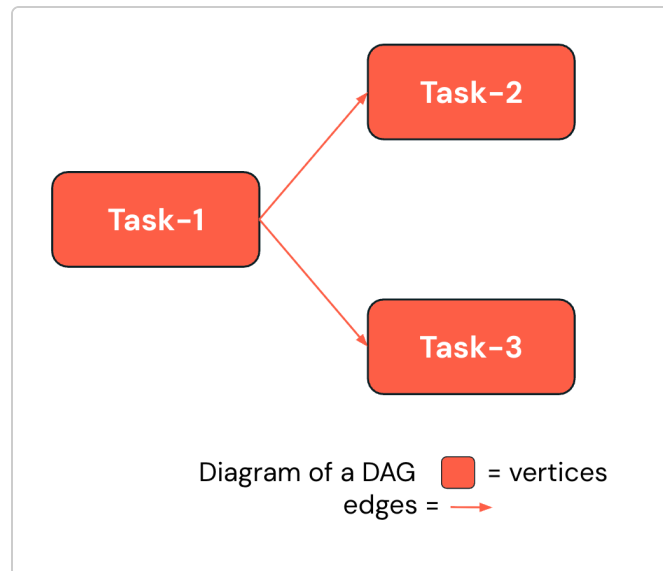
Ein wichtiges Prinzip: Jeder Task in einem Job kann seine eigene Recheninstanz zugewiesen bekommen. Tasks im selben Job können dieselbe Compute-Ressource teilen oder unterschiedliche verwenden – etwa Task 1 auf einem All-Purpose-Cluster, Task 2 auf Serverless und Task 3 auf einem dedizierten Job-Cluster. Diese Flexibilität erlaubt es, Performance und Kosten für jeden einzelnen Task gezielt zu optimieren, statt den gesamten Job auf eine einzige Kompromisslösung festzulegen.

Zur Auswahl stehen im Wesentlichen vier Compute-Kategorien:

- **Interactive Clusters** – von mehreren Nutzern gemeinsam verwendete Cluster, gut für Exploration, aber in Produktion nicht kosteneffizient, da sie auch im Leerlauf weiterlaufen.
- **Job Clusters** – werden für den Job erzeugt und nach dessen Ende automatisch beendet; dadurch spürbar günstiger, aber mit Cloud-seitiger Startzeit verbunden.
- **Serverless** – vollständig verwalteter Dienst ohne eigene Cluster-Administration, mit schnellerem Start und automatischer Skalierung.
- **SQL-Warehouse** – speziell für SQL-Abfragen, Dashboards und BI-Workloads optimiert, standardmäßig serverless mit automatischem Start/Stop.

### 4. Orchestrierung als DAG

Databricks stellt die Abhängigkeiten zwischen Tasks als **Directed Acyclic Graph (DAG)** dar – ein *gerichteter, azyklischer Graph*: gerichtet, weil jede Verbindung eine eindeutige Ausführungsrichtung hat, und azyklisch, weil es keine Rückkopplungen (Zyklen) gibt. Ein einfaches Beispiel: Task 1 muss abgeschlossen sein, bevor Task 2 und Task 3 parallel starten können. Diese Abhängigkeiten werden über die **Depends on**-Einstellung jedes Tasks konfiguriert und lassen sich über UI, REST-API, SDK oder Databricks Asset Bundles definieren.



*Ein einfacher DAG: Task 1 ist Voraussetzung für Task 2 und Task 3.*

## 5. Typische Orchestrierungsmuster

In der Praxis lassen sich die meisten Workflows auf drei wiederkehrende Muster zurückführen:

| Muster                   | Beschreibung                                                              | Typischer Anwendungsfall                                   |
|--------------------------|---------------------------------------------------------------------------|------------------------------------------------------------|
| <b>Sequenz</b>           | Tasks laufen strikt nacheinander ab.                                      | Bronze → Silver → Gold in der Medallion-Architektur        |
| <b>Trichter (Funnel)</b> | Mehrere Tasks laufen parallel und münden in einen gemeinsamen Folge-Task. | Mehrere Datenquellen werden zusammengeführt                |
| <b>Fächer (Fan-out)</b>  | Ein Task verzweigt in mehrere parallele Folge-Tasks.                      | Eine Quelle wird an mehrere nachgelagerte Systeme verteilt |

## 6. Ein Job in der Praxis: Notebook- und SQL-Task kombiniert

Ein typischer Einstieg besteht darin, einen Job mit zwei Tasks anzulegen: einem Notebook-Task, der Rohdaten in eine Bronze-Tabelle lädt, und einem SQL-Task, der eine weitere Tabelle aus einer SQL-Datei heraus befüllt. Beide Tasks lassen sich unabhängig voneinander konfigurieren (Pfad, Compute, Parameter) und anschließend über **Run now** gemeinsam starten. Nach dem Lauf zeigt die **Runs**-Ansicht den Fortschritt farbcodiert an:

- **Grün** – Task erfolgreich abgeschlossen
- **Rot** – Task fehlgeschlagen
- **Gelb** – wartend oder in Wiederholung (Retry)
- **Grau** – noch nicht gestartet, abgebrochen oder Timeout

Das folgende Beispiel zeigt, wie sich die per Job-Task erzeugten Tabellen anschließend ganz normal per SQL abfragen lassen:

```
-- Ergebnis des Notebook-Tasks prüfen
SELECT *
FROM orders_bronze
LIMIT 50;

-- Ergebnis des SQL-Tasks prüfen
SELECT *
FROM sales_bronze
LIMIT 5;
```

**Ergänzt aus der offiziellen Databricks-Dokumentation:** Lakeflow Jobs ist die aktuelle Bezeichnung für die frühere Orchestrierungskomponente „Databricks Workflows“ und bleibt inhaltlich weitgehend identisch. Die Tasks eines Jobs werden als DAG dargestellt und lassen sich über die Job-UI, die REST-API, das SDK oder Databricks Asset Bundles definieren. Neben Notebook-, SQL- und Pipeline-Tasks führt die Dokumentation unter anderem Python-Skript-Tasks auf; die vollständige Liste aller unterstützten Task-Typen findet sich im Abschnitt „Types of tasks“ der Referenzdokumentation.

Quelle: [Control the flow of tasks within Lakeflow Jobs – Databricks-Dokumentation](#)